

Instructions

- Create a folder with folder name your **roll number** in /home. Upload your .c files to that folder.
- For each question, submit one .c file, named in the format **q01.c**, **q02.c**, etc.
- Provide commented, indented code. Variables should have meaningful names.
- Marks will be awarded on the basis of how *well-written* the code is and how the program performs on closed test cases.

Questions: 10

Marks: 75

Time: 14:00–16:45

1. Student Result Determination

Marks: 3

Write a C program that takes an integer input representing a student's marks and determines their result based on the following criteria:

- Print **PASS** if the marks are between 40 and 100 (inclusive).
- Print **FAIL** if the marks are between 0 and 39 (inclusive).
- Print **INVALID** if the marks are outside the 0 to 100 range (e.g., negative numbers or numbers greater than 100).

Input Format

- A single integer representing the marks obtained by the student.

Output Format

- A single line containing one of the following strings: **PASS**, **FAIL**, or **INVALID**.

Open Test Cases

Test Case 1

Input:

75

Output:

PASS

Test Case 2

Input:

35

Output:

FAIL

Test Case 3

Input:

105

Output:

INVALID

Test Case 4

Input:

40

Output:

PASS

2. Roots of a Quadratic Equation

Marks: 8

To find the roots of a quadratic equation. You have to get the coefficients (a , b , c) from the user. The term discriminant $= b^2 - 4ac$ is known as the discriminant of a quadratic equation. It tells the nature of the roots.

- If the equation is not quadratic, print "NOT A QUADRATIC EQUATION".
- If the discriminant is greater than 0, the roots are real and different.

$$\text{root1} = \frac{-b + \sqrt{\text{discriminant}}}{2a}, \quad \text{root2} = \frac{-b - \sqrt{\text{discriminant}}}{2a}$$

- If the discriminant is equal to 0, the roots are real and equal.

$$\text{root1} = \text{root2} = \frac{-b}{2a}$$

- If the discriminant is less than 0, the roots are complex and different. Display "Imaginary Roots"

Input Format

- Three integers, one on each of three different lines (a , b , c).

Output Format

- Display the nature of the roots.
- Display the roots (to 2 decimal places) only if discriminant > 0 or discriminant $= 0$.

Open Test Cases

Test Case 1

Input:

```
1
-5
6
```

Output:

```
Real and Different Roots
root1 = 3.00
root2 = 2.00
```

Test Case 2

Input:

```
1
-4
4
```

Output:

```
Real and Equal Roots
root1 = root2 = 2.00
```

Test Case 3

Input:

```
1
2
5
```

Output:

```
Imaginary Roots
```

Test Case 4*Input:*

0
2
-4

Output:

NOT A QUADRATIC EQUATION

3. Complete and Debug: Sum of First N Natural Numbers

Marks: 6

The following program is intended to read an integer N and compute the sum of the first N natural numbers ($1 + 2 + \dots + N$) using a `while` loop. It is incomplete and also contains bugs. Complete the blanks and fix the bugs so that the program compiles and produces the correct output.

Given Code (to be completed and debugged)

```

1 #include <stdio.h>
2 int main()
3 {
4     int N, count, sum;
5     scanf(-----);
6
7     sum = 0;
8     count = 0;
9     while (count >= N);
10    {
11        -----;
12        -----;
13    }
14    print("Sum = -----");
15 }
```

Listing 1: Incomplete/buggy code – fill in the blanks and fix the bugs

Input Format

- A single integer N .

Output Format

- A single line: Sum = <value>

Open Test Cases

Test Case 1

Input:

Output:

Test Case 2

Input:

Output:

Test Case 3

Input:

Output:

4. Simple Calculator using switch-case

Marks: 10

Write a C program that reads two integers and an operator character (+, -, *, /, %) and performs the corresponding arithmetic operation using a `switch` statement.

Handle division by zero and modulo by zero separately, and handle any operator that is not one of the five listed above.

Rules:

- For '/' and '%', if the second number is 0, print "MATH ERROR"
- For any operator other than +, -, *, /, %, print "INVALID OPERATOR"
- Division result should be printed as a float with 2 decimal places
- All other results should be printed as integers

Input Format

- Line 1: An integer a
- Line 2: An integer b
- Line 3: A single character operator

Output Format

- A single line with the result, or "MATH ERROR" / "INVALID OPERATOR"

Open Test Cases

Test Case 1

Input:

```
10
3
+
```

Output:

```
13
```

Test Case 2

Input:

```
10
3
/
```

Output:

```
3.33
```

Test Case 3

Input:

```
10
0
%
```

Output:

```
MATH ERROR
```

Test Case 4

Input:

10
3
~

Output:

INVALID OPERATOR

Test Case 5*Input:*

5
3
-

Output:

2

5. Vending Machine Change Calculator

Marks: 10

Write a C program that simulates a simple vending machine. It reads the price of an item and the amount inserted by the user (both integers, in rupees). Using if-else and the modulo/division operators:

- If amount inserted $<$ price: print "INSUFFICIENT FUNDS, need X more" where X is the shortfall.
- If amount inserted $=$ price: print "EXACT PAYMENT, ENJOY!"
- If amount inserted $>$ price: compute the change and print how many 100, 50, 20, 10, 5, and 1 rupee notes/coins to return, greedily (largest denomination first), one per line in the format "DENOMINATION x COUNT". Skip printing any denomination whose count is 0.
- Handle negative inputs gracefully by printing "INVALID INPUT".

Note: Do this WITHOUT using an array or loop — only sequential if statements using % and / repeatedly on the remaining change (this is meant to make you appreciate why loops/arrays will be useful later).

Input Format

- Line 1: price (integer)
- Line 2: amount inserted (integer)

Output Format

- As described above

Open Test Cases

Test Case 1

Input:

```
37
100
```

Output:

```
50 x 1
10 x 1
1 x 3
```

Test Case 2

Input:

```
50
50
```

Output:

```
EXACT PAYMENT, ENJOY!
```

Test Case 3

Input:

```
80
60
```

Output:

```
INSUFFICIENT FUNDS, need 20 more
```


6. Bitwise Playground: Check Odd/Even and Swap Case

Marks: 6

Part A: Read an integer and determine if it is odd or even using the bitwise AND operator (&) instead of the modulo operator (%).

Part B: Read a single alphabet character and toggle its case (lowercase to uppercase or uppercase to lowercase) using appropriate bitwise operator, instead of using if-else or built-in functions.

Input Format

- Line 1: An integer n
- Line 2: A single alphabet character (upper or lower case)

Output Format

- Line 1: ODD or EVEN
- Line 2: The character with case toggled, if line 2 of the input is an alphabet letter (upper or lower case).
- If line 2 of the input is **not** an alphabet letter, print INVALID CHARACTER on line 2 instead — do not attempt to toggle its case.

Open Test Cases

Test Case 1

Input:

```
7
g
```

Output:

```
ODD
G
```

Test Case 2

Input:

```
10
Z
```

Output:

```
EVEN
z
```

Test Case 3

Input:

```
7
5
```

Output:

```
ODD
INVALID CHARACTER
```

7. Rock Paper Scissors

Marks: 10

Two students are playing the classic Rock-Paper-Scissors game.

The choices are:

- R – Rock
- P – Paper
- S – Scissors

Write a C program that reads the choices of Player 1 and Player 2 and determines the winner using `switch` statements.

Rules:

- Rock defeats Scissors.
- Scissors defeats Paper.
- Paper defeats Rock.
- If both players choose the same option, print `DRAW`.
- If either player enters an invalid choice, print `INVALID INPUT`.

Note: The input characters are case-sensitive. Assume that the user enters uppercase letters only.

Input Format

- Line 1: A character representing Player 1's choice (R, P or S)
- Line 2: A character representing Player 2's choice (R, P or S)

Output Format

Print one of the following: `PLAYER 1 WINS`, `PLAYER 2 WINS`, `DRAW`, `INVALID INPUT`

Open Test Cases

Test Case 1

Input:

R
S

Output:

PLAYER 1 WINS

Test Case 2

Input:

P
S

Output:

PLAYER 2 WINS

Test Case 3

Input:

R
R

Output:

DRAW

Test Case 4*Input:*

X
P

Output:

INVALID INPUT

Test Case 5*Input:*

P
R

Output:

PLAYER 1 WINS

8. Hailstone Sequence (Collatz Conjecture)

Marks: 6

The Collatz Conjecture states that starting from any positive integer n , if you repeatedly apply the following rule, you will eventually reach 1:

- If n is even, the next term is $n/2$.
- If n is odd, the next term is $3n + 1$.

You have to read a starting number n from the user, generate the entire sequence (called the *Hailstone Sequence*) until it reaches 1, and print it. Also count the number of **steps** taken to reach 1 (the starting number itself does not count as a step).

- If $n \leq 0$, print "INVALID INPUT" and do nothing else.
- If $n = 1$, the sequence is just 1 and the number of steps is 0.

Use a **while** loop to generate the sequence.

Input Format

- A single integer n .

Output Format

- If $n \leq 0$: print "INVALID INPUT".
- Otherwise, print the full sequence on one line, numbers separated by a single space.
- On the next line, print "Steps = X" where X is the number of steps.

Open Test Cases

Test Case 1

Input:

Output:

Test Case 2

Input:

Output:

Test Case 3

Input:

Output:

9. Reverse Digits and Check Palindrome

Marks: 10

Write a program that reads an integer N from the user, reverses its digits, and determines whether N is a *palindrome* (reads the same backward as forward, like 121).

- You must use a **for** loop to extract and reverse the digits (not **while** or **do-while**). Do not use arrays or strings.
- If N is negative, reverse the digits of its magnitude and print the reversed value with a negative sign (e.g. $-121 \rightarrow -121$). A negative number is never a palindrome, regardless of its digits.
- If N has trailing zeros (e.g. 120), those become leading zeros when reversed and are simply dropped, since the reversed value is printed as a number (e.g. $120 \rightarrow 21$, not 021).

Input Format

- A single integer N .

Output Format

- Line 1: **Reversed = X**, where **X** is N with its digits reversed.
- Line 2: **PALINDROME** if N is a non-negative number that reads the same forward and backward, otherwise **NOT PALINDROME**.

Open Test Cases

Test Case 1

*Input:**Output:*

Test Case 2

*Input:**Output:*

Test Case 3

*Input:**Output:*

10. Pingala Sequence (First N Terms)

Marks: 6

Write a program to generate and print the first N terms of the Pingala sequence (0, 1, 1, 2, 3, 5, 8, 13, ...), where each term (from the third onward) is the sum of the two preceding terms.

You must use a **single for** loop to generate and print all N terms. Do not use arrays, nested loops, or **while/do-while**.

Input Format

- A single integer N , with $1 \leq N \leq 90$ (this range keeps every term well within what a 64-bit integer can hold).

Output Format

- If $N \leq 0$: print "INVALID INPUT".
- Otherwise, print the first N terms on a single line, separated by a single space.

Open Test Cases

Test Case 1

Input:

1

Output:

0

Test Case 2

Input:

2

Output:

0 1

Test Case 3

Input:

5

Output:

0 1 1 2 3